

CONTENT MODERATION COMMAND CENTER (CMCC)

Technical Architecture & Phased Development Plan

1. EXECUTIVE SUMMARY

CMCC is a WordPress plugin that consolidates all user-generated content moderation into a single, visual “command center.” It replaces the fragmented native moderation screens (comments, pending posts, media, users, form entries) with a unified queue, real-time analytics heatmaps, a spam firewall, and direct moderation actions—including user deactivation.

BUSINESS MODEL

- CMCC Core (v1): Free. Builds user base.
- CMCC Pro (v2): Premium plugin (one-time purchase or yearly) sold from our site.
- CMCC AI Service: Cloud API, charged by monthly usage credits. Starts at \$9/month.
- Pro plugin requires an active API key from us.

PHASED APPROACH

- Phase 1: Build and release v1 on WordPress, Strapi, StoryBok, Wix and Shopify.
- Phase 2: If adoption is strong, develop v2 Pro + AI backend simultaneously.

2. V1 – CMCC CORE (FREE PLUGIN)

Target: WordPress, Strapi, Storyblok, Wix and Shopify addon

License: GPLv2+

2.1 UNIFIED MODERATION QUEUE

A single admin page (“Moderation Center”) with a sortable, filterable list of all pending/spam/flagged items.

SUPPORTED CONTENT TYPES

- Comments (status: pending, spam, held)
- Posts/Pages/CPTs (status: pending, approved)
- Media attachments (attached, unattached or flagged)
- User accounts (pending approval, or new with no published content)
- WooCommerce product reviews (if WooCommerce active)
- bbPress topics/replies (if bbPress active)
- BuddyPress activity/group posts
- Form entries: Gravity Forms, WPForms, Contact Form 7 (status “unread” or custom moderation flag)

LIST COLUMNS

- Checkbox (for bulk actions)
- Type icon (comment bubble, file, user, etc.)
- Title/Excerpt (clickable to edit/view)
- Author (avatar + name, linked to user profile)
- Date submitted
- Status (pending/spam/flagged)

- Spam score (if Akismet active)
- Quick action links: Approve | Reject | Mark Spam | Defer

BULK ACTIONS (DROPDOWN + APPLY)

- Approve All (publishes, approves comment, etc.)
- Move to Trash
- Mark as Spam
- Deactivate User Accounts (for user-submitted items, based on admin roles and permissions if the internal user has been granted the permission by the site admin.)
- Export selected to CSV

IMPLEMENTATION

- Custom database table: wp_cmcc_queue
- Hooks capture new items: wp_insert_comment, save_post, add_attachment, user_register, gform_after_submission, wpcf7_before_send_mail, etc.
- Items removed from queue when moderated (moved to history table for audit).
- Admin page uses WP_List_Table or custom React-based table.

2.2 SPAM FIREWALL (RULE-BASED)

Pre-submission checks that either block or flag content.

CONFIGURABLE RULES (SETTINGS PAGE)

- Max number of links in content (default 3)
- Blacklisted keywords (exact + partial match)
- Blacklisted IP addresses / email domains
- Country blocking (using free GeoLite2 database)
- Content submitted too quickly (hidden honeypot field + time check)
- Duplicate content detection: hash the content (md5) and compare against existing comments/posts in database.

ACTIONS WHEN A RULE TRIGGERS

- “Flag for review” (sets item to pending, adds “firewall flag” tag)
- “Silently discard” (block submission with no notice)
- “Send to spam” (only for comments)

IMPLEMENTATION

- Hook into: pre_comment_approved, wp_insert_post_data, wp_handle_upload_prefilter, registration_errors.
- Settings stored in wp_options as array.

2.3 USER REPUTATION & BREACH TRACKER

Every user gets a reputation score (integer, stored in usermeta). Score decays over time.

SCORING RULES

- Approved item: +1
- Rejected/spam item: -2
- User account deactivated by moderator: -10
- After 30 days of inactivity, score drifts toward 0 by 1 point every 7 days (WP Cron).

DASHBOARD WIDGET “TOP OFFENDERS”

- Lists users with score below -5.
- One-click “Deactivate” sets user role to cmcc_suspended (custom role with no capabilities). Adds a user note.

BREACH LOG

- Stored in wp_cmcc_user_breaches table: user_id, timestamp, reason, moderator.

2.4 ANALYTICS & HEATMAPS

Visual layer integrated with the queue.

DATA COLLECTION

- Custom table: wp_cmcc_events logs every moderation-related event.
- Columns: id, timestamp, type, item_id, action (created/approved/spammed/trashed), user_id, content_type.

ANALYTICS DASHBOARD (SUB-PAGE)

- Heatmap: Y-axis = day of week, X-axis = hour (0-23). Color intensity = count of items pending/spam created. Clickable to drill into queue.
- Spam ratio gauge: today's spam count vs total comments.
- Content-type breakdown pie chart.
- Moderator performance bar chart: approvals/trashes per moderator (past week).
- Anomaly detection: rule-based thresholds (e.g., >30 new pending items in an hour) highlight heatmap cell red and trigger notification.

TECHNICAL

- All charts rendered client-side using Chart.js with heatmap plugin.
- PHP serves JSON via admin-ajax endpoints.

2.5 AUDIT LOG & ACTIVITY FEED

Separate tab "Activity Log" showing chronological feed of every moderation action.

COLUMNS

- Timestamp, Moderator, Action, Item type, Item title (link), Previous status, New status.

FEATURES

- Searchable, filterable by date and moderator.
- Reversals: "Undo" link for recent actions (within 5 minutes).
- Auto-prune log entries older than 90 days (configurable).

STORAGE

- Table: wp_cmcc_audit_log

2.6 NOTIFICATIONS

- Admin email when pending queue exceeds threshold (default 50).
- "Daily Digest" email (WP Cron) listing totals.
- Dashboard alert bar when anomalies detected.

2.7 MULTISITE COMPATIBILITY

- Network activation supported.
- Super-admin sees "Network Moderation" view aggregating all subsites' pending items (read-only overview, click to go to each subsite's CMCC).
- Tables created per subsite (use \$wpdb->prefix).

2.8 CUSTOM TABLE SCHEMAS

```
CREATE TABLE wp_cmcc_queue (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  content_type VARCHAR(20) NOT NULL,
```

```

original_id BIGINT UNSIGNED NOT NULL,
blog_id BIGINT UNSIGNED DEFAULT 1,
status VARCHAR(20) NOT NULL DEFAULT 'pending',
spam_score FLOAT DEFAULT 0,
author_id BIGINT UNSIGNED,
date_gmt DATETIME NOT NULL,
title VARCHAR(255),
excerpt TEXT,
metadata JSON,
INDEX (content_type, status),
INDEX (author_id),
INDEX (date_gmt)
);

CREATE TABLE wp_cmcc_events (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
timestamp DATETIME NOT NULL,
content_type VARCHAR(20),
action VARCHAR(20),
item_id BIGINT UNSIGNED,
user_id BIGINT UNSIGNED,
blog_id BIGINT UNSIGNED DEFAULT 1,
INDEX (timestamp),
INDEX (content_type, action)
);

CREATE TABLE wp_cmcc_audit_log (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
timestamp DATETIME NOT NULL,
moderator_id BIGINT UNSIGNED,
content_type VARCHAR(20),
item_id BIGINT UNSIGNED,
action VARCHAR(20),
old_status VARCHAR(20),
new_status VARCHAR(20),
note TEXT,
INDEX (timestamp),
INDEX (moderator_id)
);

CREATE TABLE wp_cmcc_user_breaches (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
user_id BIGINT UNSIGNED,
timestamp DATETIME,
reason VARCHAR(100),
moderator_id BIGINT UNSIGNED,
INDEX (user_id)
);

```

2.9 ADMIN UI/UX

- Top-level menu icon: radar/screen icon.
- Submenus: Queue, Analytics, Activity Log, Settings.
- Queue page: WP_List_Table (or React-based for modern feel). AJAX quick actions.
- Settings: Tabs for general, spam firewall rules, notifications.
- Dark mode CSS toggle stored in user meta.

=====

3. V2 – CMCC PRO (PAID PLUGIN) + AI SERVICE

Target: Sold via our own website (Easy Digital Downloads/Freemius).
Requires: Active CMCC AI API key (subscription).
Pro plugin is a drop-in add-on that extends the free core.

3.1 AI MODERATION ENGINE

When a new item enters the queue, the plugin optionally sends it to the CMCC AI API for analysis.

PROCESS

1. Hook into queue insertion.
2. If AI pre-screening enabled, make async HTTP request to our API.
3. Store the AI decision in `cmcc_queue.ai_decision` JSON column.
4. In the queue list, show colored badge:
 - "AI: Approve (95%)" (green)
 - "AI: Reject (88%)" (red)
 - "AI: Flag" (yellow)
5. Hovering shows the explanation.

AI MODEL & PROMPT DESIGN (ON OUR BACKEND)

- Text content: OpenAI GPT-4o-mini / Claude 3 Haiku.
- System prompt: "You are a strict content moderator for a website in the {site_category} niche. Review the following {content_type} and decide if it should be approved, rejected, or flagged for manual review. Consider spam, abuse, self-promotion, off-topic, and profanity. Respond only with a JSON object containing decision, confidence (0-1), reason (under 120 chars), spam_likelihood (0-100)."
- Image content: Vision model (GPT-4o or Claude) to detect NSFW, violence, hate symbols, text overlays.

3.2 AUTO-MODERATION RULES (PRO)

Settings page to define thresholds:

- "If AI confidence $\geq$ 98% and decision is 'approve', auto-approve"
 - "If decision is 'reject' and confidence $\geq$ 90%, auto-trash"
 - "If decision is 'flag', always keep in queue"
 - Content-type-specific overrides
- All auto-actions logged in audit log with moderator as "CMCC AI".

3.3 IMAGE MODERATION

- On media upload, plugin sends image URL to AI API's image moderation endpoint.
- Plugin can optionally blur the image until approved.
- Auto-reject if AI scores NSFW > threshold.

3.4 BATCH AI REVIEW

- Button in queue: "Run AI Review on all pending items".
- Uses WP Cron to process in batches of 10 items per minute.
- Shows progress bar.

3.5 AI-DRIVEN USER RISK SCORING

- AI evaluates user's last 10 contributions for toxicity/spam patterns.
- Score stored as user meta.
- High-risk users auto-flagged on every new submission.

3.6 MULTI-LANGUAGE SUPPORT

- Plugin detects WP locale, translates UI strings.
- AI service accepts language parameter, tailors moderation accordingly.

3.7 INTEGRATION WITH CMCC AI SERVICE

- Settings section for API Key, Endpoint URL, Test Connection button.
- All API calls POST JSON, authenticated via Bearer token.
- Endpoint: <https://api.cmcc.ai/v1>

4. CMCC AI SERVICE – CLOUD BACKEND & PRICING

A simple, scalable REST API that brokers AI moderation requests to underlying AI providers while managing billing, rate limits, and key validation.

4.1 API ENDPOINTS

BASE URL: <https://api.cmcc.ai/v1>

/moderate/text POST Moderate a text item (comment, post, form entry)

/moderate/image POST Moderate an image (URL)

/batch POST Moderate multiple items at once (up to 10)

/usage GET Return current plan usage (calls remaining)

REQUEST BODY (TEXT EXAMPLE)

```
{
  "content_type": "comment",
  "text": "Check out my free viagra site...",
  "language": "en",
  "site_category": "health",
  "metadata": { "author_id": "123", "ip": "..." }
}
```

RESPONSE

```
{
  "id": "req_abc123",
  "decision": "reject",
  "confidence": 0.97,
  "reason": "Pharmaceutical spam",
  "spam_likelihood": 98,
  "tokens_used": 120
}
```

4.2 AUTHENTICATION & KEY MANAGEMENT

- Upon subscribing (via Stripe), user gets a unique API key.
- Key associated with a plan tier and monthly call quota.
- All requests include header: Authorization: Bearer <api_key>
- Rate limits: 10 requests/second, 1000/hour (configurable per plan).
- Keys revocable/regenerable from user's account dashboard.

4.3 PRICING PLANS (SUBSCRIPTION, MONTHLY)

STARTER \$9/month 5,000 calls \$0.002/call overage

Text moderation only

PROFESSIONAL \$29/month 20,000 calls \$0.0015/call overage
+ Image moderation, batch

BUSINESS \$99/month 100,000 calls \$0.001/call overage
+ Priority support, custom models

ENTERPRISE Custom Custom Dedicated, on-prem option

- Upgrade/downgrade anytime.
- Unused calls roll over up to 1 month.
- Pro plugin itself: one-time fee (\$49–\$79 per site) or yearly (\$29/year).

4.4 IMPLEMENTATION STACK

- API Layer: Python FastAPI (recommended for async and validation).
- Queue: Redis + BullMQ or simple database queue for batch requests.
- AI Provider: Abstraction layer supporting OpenAI (GPT-4o-mini, GPT-4o) and Anthropic (Claude 3 Haiku). Primary: GPT-4o-mini for text.
- Database: PostgreSQL for accounts, API keys, usage logs.
- Billing: Stripe for subscriptions, webhooks to update plan/quota.
- Hosting: Cloud Run or VPS.

4.5 SECURITY & PRIVACY

- All data in transit encrypted via HTTPS.
 - Content not stored beyond API call log; logs retained 30 days, then purged.
 - Zero-log mode option (Enterprise plan).
 - GDPR compliance: Data processing agreement available.
-

Multi-Platform Core

Goal: Build and release free “CMCC Core” apps for Strapi, Storyblok, Wix, and WordPress simultaneously. Each app delivers:

- Unified moderation queue for all user-generated content on that platform
- Spam firewall (rule-based, adapted to platform)
- Visual analytics (heatmaps, charts)
- User reputation tracking
- Audit log

All apps will share a common design system and user experience. In v2, they all connect to the **CMCC AI Cloud** for automated moderation (paid add-on).

Platform-Specific Implementations

1. WordPress – CMCC Core Plugin

- **Type:** Native PHP plugin (WordPress.org)
- **Integration:** Hooks into WP actions/filters, uses custom database tables
- **Queue items:** Comments, posts (pending), media, users, WooCommerce reviews, form entries (Gravity Forms, WPForms, etc.)
- **Analytics:** Chart.js via admin-ajax, WordPress heartbeat for real-time updates
- **Firewall:** Pre-comment, pre-post, pre-upload hooks
- **User reputation:** User meta, custom suspended role
- **Packaging:** Standard WP plugin zip

2. Strapi – CMCC Plugin (Admin Panel Extension)

- **Type:** Strapi plugin (Node.js, React admin panel extension)

- **Integration:** Uses Strapi's plugin SDK; accesses core APIs to list/moderate content
- **Queue items:** All collection types marked as "moderated" (e.g., comments, user-submitted posts, reviews, form submissions); custom life-cycle hooks via Strapi's content-type lifecycles
- **Firewall:** Strapi middlewares that check IP, keywords, link count, honeypot before persisting
- **Analytics:** Custom admin page with Chart.js; data stored in a dedicated content-type or database table
- **User reputation:** Strapi's user model extended with `reputation` field; "suspend" via role change
- **Packaging:** Strapi plugin installable via npm or marketplace

3. Storyblok – CMCC App (Custom App in Storyblok App Directory)

- **Type:** React app rendered inside Storyblok's App Container (Storyblok App SDK)
- **Integration:** Uses Storyblok Management API to fetch and moderate content entries, datasource entries, and form submissions (if using Storyblok Forms)
- **Queue items:** Unpublished entries, flagged entries (via custom field or workflow stage), comments (if using external service like Disqus, via proxy)
- **Firewall:** Implemented via Storyblok webhooks that trigger on publish attempt; an external serverless function checks rules and rejects via API (or a Storyblok "moderation" app hook)
- **Analytics:** Built into the app UI using Chart.js; data persisted via Storyblok datasource or external storage
- **User reputation:** Storyblok doesn't have native users; could use a list of external user IDs and maintain scores in a datasource; "suspend" is manual or via API
- **Packaging:** App submitted to Storyblok App Directory, installable by any space

4. Wix – CMCC Wix App (Velo)

- **Type:** Wix App built with Velo (Wix's development platform) and Wix Dashboard App SDK
- **Integration:** Uses Wix APIs (Comments, Blog, Forms, Members) to retrieve all pending/unapproved content
- **Queue items:** Blog comments (pending), forum posts, member-submitted content (via Wix CMS if used), form submissions (Wix Forms)
- **Firewall:** Velo hooks (`onComment` , `onFormSubmit`) to run checks and block or flag
- **Analytics:** Dashboard page with embedded analytics using a custom Velo chart library or embedded Chart.js iframe
- **User reputation:** Wix Members custom fields for score; deactivation by blocking member via API
- **Packaging:** App submitted to Wix App Market

Shared Architecture for Multi-Platform Consistency

To keep development efficient and UI identical across platforms, we will create:

Shared NPM Packages (private or open source):

- `cmcc-queue-renderer` : React component for the moderation queue list (sortable, filterable, bulk actions). Used in Strapi, Storyblok, and Wix (where React/Velo allows).
- `cmcc-charts` : React component for analytics dashboards (heatmap, pie, bar) powered by Chart.js. Platform-agnostic.
- `cmcc-firewall-rules` : Pure JavaScript rule engine (keyword matching, link counting, time check, duplicate hash) that can be used on any Node.js/Velo environment. For PHP (WordPress), the rules will be rewritten in PHP (shared logic, separate implementation).
- `cmcc-reputation` : Scoring algorithm and decay logic, also pure JS/PHP dual implementation.

This means:

- The Strapi plugin is a thin wrapper that mounts `cmcc-queue-renderer` and `cmcc-charts` inside Strapi's admin panel, communicating with Strapi's REST API.
- The Storyblok app is a React app that imports those components and uses the Storyblok JS SDK.
- The Wix app will use similar React components via Wix's custom element support (if allowed) or replicate with Velo's native UI.
- The WordPress plugin will implement a similar UI using Backbone/jQuery or React (via `@wordpress/element`) but will still draw from the same UX blueprint.

No in house server is required for v1.

Every CMCC Core app for v1 runs entirely within the user's existing infrastructure or the platform's own cloud environment. We do not host anything, and there are no centralised servers that we maintain. Here's how it works for each platform:

WordPress – The plugin is pure PHP and JavaScript, installed directly on the user's WordPress server. All data (queue, events, audit logs) is stored in custom database tables on that same server. No external calls, no external database.

Strapi – The plugin is a Strapi plugin that lives inside the user's Strapi project. It adds collections (or custom tables) to the Strapi database, and all moderation logic runs on the Strapi server. No external dependencies.

Wix – The Wix app uses Wix's built in Velo backend (Node.js functions hosted by Wix). All data is stored in Wix's built in database collections. The app runs entirely inside Wix's infrastructure – we don't run anything.

Storyblok – The app is a client side React application that gets rendered inside Storyblok's App Container. The moderation queue and analytics data are persisted inside the user's Storyblok space using a dedicated datasource. The spam firewall for Storyblok will be implemented via a serverless function that the user deploys themselves (we provide the open source code, which they can deploy for free on Vercel, Netlify, or Cloudflare Workers). Alternatively, if Storyblok's App SDK later supports publish intercept hooks, the firewall can be moved entirely inside the app, removing any external call. In either case, we do not host the serverless function – the user does, using their own free tier account. This keeps v1 completely self contained without us running any central service.

v1 Development Plan (Simultaneous)

Team setup: One AI-assisted developer for each platform, plus a core component developer.

Timeline: 8–10 weeks

Week 1-2: Foundation

- Build `cmcc-firewall-rules`, `cmcc-reputation`, and draft UI components.
- Set up development environments for all platforms.

Week 3-4: Core Backend for Each Platform

- **WordPress:** Create plugin, DB tables, hooks for queue, firewall integration.
- **Strapi:** Bootstrap plugin, define content-type for queue events, add middleware.
- **Storyblok:** Set up app scaffolding, Management API integration, webhook listener (serverless).
- **Wix:** Scaffold Wix App with Dashboard page, connect to Wix APIs, implement spam checks in `onComment` / `onFormSubmit`.

Week 5-6: Unified Queue & Moderation UI

- Build the moderation queue page in each platform using either shared React components or native equivalents.
- Implement bulk actions, single-item moderation, user deactivation where possible.

Week 7-8: Analytics & Audit Log

- Integrate `cmcc-charts` into each platform's analytics page.
- Implement events logging and audit trail.
- Create notifications (email, in-app) – platform-appropriate.

Week 9-10: Polish, Testing, & Submission

- Cross-platform testing (e.g., spam a Strapi site, check analytics).
 - Prepare marketing materials (common landing page, platform-specific app listings).
 - Submit to marketplaces: WordPress.org, Strapi Market, Storyblok App Directory, Wix App Market.
 - Add “Buy Me a Coffee” / donation links inside each app.
-

Phase 2 – v2 Pro + AI Service

Pre-condition: At least 500 active installs and positive feedback, or a clear signal of demand.

Duration: 8-10 weeks parallel for plugin add-on and cloud API.

Plugin Pro Add-On:

Add settings page for API key and features.

AI pre-screening integration (display badge, action).

Auto-moderation rules UI and logic.

Batch AI with WP-Cron progress indicator.

Image moderation hook and blur functionality.

User risk score request on profile.

AI Service:

Build API with text moderation endpoint, key management, usage tracking.

Integrate with Stripe for subscription plans.

Developer dashboard (account, usage, billing).

Basic moderation prompt engineering and testing.

Transition to v2: AI Service Integration

All v1 apps will be designed with an optional “AI Settings” tab where the user can enter a CMCC API key. This tab will be hidden or show a “Coming Soon” message in v1. In v2, we release a **CMCC AI Cloud** and a **CMCC Pro** activation key that unlocks the tab. The AI service integration will be handled via a shared API client (`cmcc-ai-client`) that each platform’s plugin/app will use. The AI backend remains identical to the previously described FastAPI + Stripe system.

That’s a crucial design question, and the short answer is:

We can achieve a near-identical UX (workflows, interactions, layout) across all platforms, but the visual UI (colors, typography, spacing, component styling) must adapt to each platform’s native design system.

Forcing exactly the same pixel-perfect look everywhere would violate platform conventions and hurt adoption. But a **consistent user experience** — the same mental model, the same queue structure, the same moderation actions — is absolutely possible and highly desirable. Here’s how to do it.

1. Why a Fully Identical UI Is a Bad Idea

Platform	Native Design System	User Expectation
WordPress	wp-admin (jQuery UI / Calypso-ish)	Users expect panels, tabs, and traditional admin styling. A fully custom SPA feels alien.
Strapi	Strapi Design System (Buffet.js)	Users expect a clean, modern, left-nav-oriented React admin.
Storyblok	Storyblok Design System (carbon)	Apps sit inside an iframe; they should mirror Storyblok’s look, typography, and spacing.
Wix	Wix Design System (Stylable, Velo)	Dashboard apps must follow Wix’s component library and responsive patterns.

If CMCC appeared in WordPress looking like a Strapi plugin, users would assume it’s a third-party clone and might distrust it. Conversely, a wp-admin-style interface inside Strapi would feel broken. **Platform consistency is trust.**

2. The Sweet Spot: Shared UX + Platform Adapted UI

UX (interaction patterns, information architecture, logic):

- Same moderation queue layout (list with columns, bulk actions, quick filters)
- Same analytics page structure (heatmap, gauge, pie chart)
- Same settings flow (tabs for general, firewall, notifications)

- Same terminology and feedback messages

UI (visual skin):

- Colors, button styles, form elements, fonts, spacing: **all derived from the platform's design system**
- The shared functional components are headless; presentation is injected per platform

This means you can take a moderator who knows CMCC on WordPress, drop them into a Strapi project, and they'll immediately know where everything is — even though it looks native to Strapi. That's a powerful selling point for agencies managing multiple platforms.

3. How to Build This in Practice

We create a **single, shared component library** (`cmcc-ui-core`) using React (since Strapi, Storyblok, Wix all support React; WordPress can use `@wordpress/element` which is React based). This library:

- Contains all the functional UI logic (queue table, chart containers, bulk actions, filters)
- Exposes a **theme provider** / design token context
- Accepts a `platform` prop or uses a `design` token injection

Then, for each platform, we build a thin wrapper that:

1. Imports the shared components
2. Supplies the correct design tokens (colors, spacing, border radii, font stacks) pulled from the platform's own CSS variables or design system
3. Wraps everything in the platform's layout primitives (e.g., `SlotFill` in WordPress, Strapi's `Layout` component, Storyblok's app shell)

The shared components never import CSS directly; they use a `ThemeContext` and render generic HTML elements with class names that the wrapper styles. This is essentially a **headless + theming** architecture, similar to Radix UI or Chakra UI v2.

Option for WordPress:

We could also implement the UX using plain React and load it inside a custom admin page, ignoring wp-admin styling entirely. But that might feel disconnected. A better compromise is to keep the outer shell wp-admin (menu, notices, footer) and only make the queue/analytics pages modern and self-consistent, using a minimal CSS reset that doesn't clash with wp-admin. Many popular plugins (e.g., Yoast SEO, WooCommerce) use this hybrid approach: wp-admin frame, but custom clean UI inside.

4. Platform Specific Adaptation Details

WordPress

- Shell: WordPress admin menu and admin bar remain untouched.
- Our pages: Built with React (`@wordpress/element`), using a set of design tokens that match the WordPress color palette (`#007cba` , `#f0f0f1` , etc.) and the standard WP font (`apple-system` , `BlinkMacSystemFont`).
- We can incorporate WordPress UI components (`Button` , `Modal` , `Notice`) where possible to feel native, but our shared analytics widgets will use a stripped-down, clean look that doesn't break the visual rhythm.
- A WordPress-specific CSS file will tweak the shared components' spacing to feel comfortable in the wp-admin content area.

Strapi

- Shell: Strapi's left-hand menu; our plugin adds a "Moderation Center" item inside the plugins section.
- Content area: Fully native Strapi UI using the Strapi Design System (Buffet.js). Our shared components will be wrapped with Strapi's `Box` , `Typography` , and `Button` components, making them indistinguishable from core Strapi features.
- The theme tokens will map to Strapi's design variables (e.g., `neutral800` , `primary600`).

Storyblok

- Shell: The app sits inside the Storyblok app container (an iframe). Storyblok provides a CSS class namespace and a small SDK for styling.
- Our app will include the Storyblok Design System components or at least match their color and spacing values. Because it's an iframe, we can use any React framework, but we must ensure it looks like it belongs in the Storyblok editor (e.g., same background, same button styles).
- A Storyblok specific theme adapter will be used.

Wix

- Shell: Wix Dashboard apps have a specific layout (top bar, side navigation). We'll use Wix's own component library (Stylable) to render our queue and analytics. Since Wix supports React, we can import our headless components but render them with Wix's design system components (wix style react or similar).
- The theme adapter will map our tokens to Wix's palette.

5. Balancing Branding vs. Native Fit

We want CMCC to be recognisable as a product, not just a generic tool. We can achieve subtle branding that works across platforms:

- A **small CMCC logo/icon** in the corner of the main screen, or in the footer.
- Consistent **terminology** and **layout structure** (e.g., the heatmap is always top right, the queue always has a “quick action” toolbar).
- A **distinctive “command center” color accent** (e.g., a radar green or electric blue) used sparingly for things like notification badges, AI badges, and the main menu icon. This accent can be applied consistently across all platforms without clashing with their design systems — it’s just a highlight color.

Users will come to recognise “that green dot means CMCC is active,” regardless of the platform they’re on.

6. Recommendation for v1 Implementation

Build a **single monorepo** with:

1. packages/cmcc-core – platform agnostic React hooks, state management, queue logic, firewall rule engine (pure functions), analytics data processing.
2. packages/cmcc-ui – headless React components (queue table, chart containers, etc.) that use a theme context.
3. platforms/wordpress , platforms/strapi , platforms/storyblok , platforms/wix – each contains a thin adapter that wires cmcc-ui into the platform’s shell, supplies the theme tokens, and handles platform specific data fetching (e.g., WP REST API, Strapi API, Wix Velo APIs).

This architecture lets a single AI agent (or a small team) build the core once, then rapidly adapt it for each platform — the UX remains identical, the UI fits natively, and maintenance stays low.

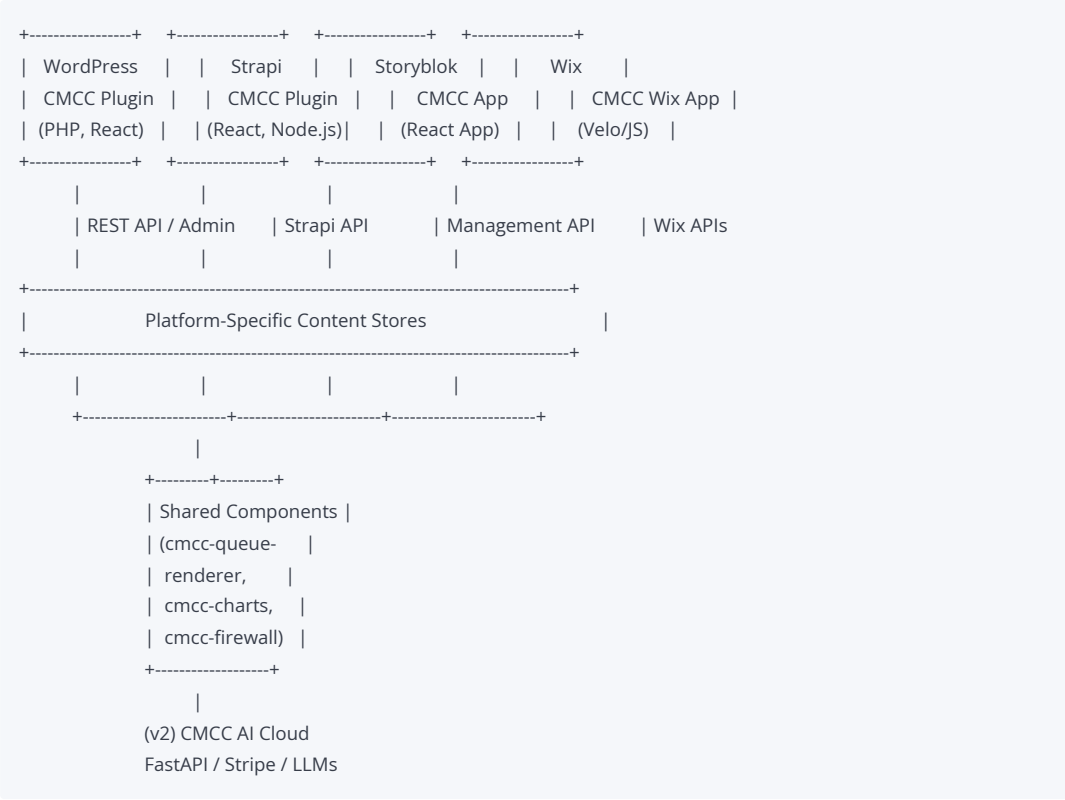
Would you like me to outline the exact file structure and theming token specification for that shared component library?

Monetization Summary

Product	Price	Revenue Model
CMCC Core	Free	Donations, builds user base
CMCC Pro (plugin)	\$49 one-time / \$29/yr	Upfront or annual plugin license
CMCC AI Service	\$9-\$99+/month	Recurring subscription (margin from AI API costs)

Example profitability: If we resell GPT-4o-mini calls at a 3-5x markup, the margin is healthy. At scale, we can negotiate volume pricing with OpenAI or switch to self-hosted LLMs.

New Technical Architecture Diagram (Textual)



Unique Value Proposition (Cross-Platform)

No other moderation tool provides a **single, consistent command center** across these diverse platforms. Site owners using a headless stack (Strapi + Storyblok) and traditional WordPress or Wix now have one familiar interface to moderate all user content, with analytics that unify their entire web presence. This becomes an essential tool for agencies managing multiple client sites on different tech stacks.

Launch: Sell CMCC Pro via our website (using Freemius/EDD). Offer a free 7-day trial of the AI service. Market to existing CMCC Core users with in-dashboard notice.